Phase 1: JavaScript Mastery - Module 2: JavaScript Fundamentals

> Let's start module 2: Javascript Fundamentals and the first topic is Hello World program

Module 2: JavaScript Fundamentals

Topic 1: Hello World Program

The traditional first program in any programming language is **Hello World**. Its purpose is to verify that the language setup is working correctly and to introduce the basic syntax.

---

1. Hello World in JavaScript

Method 1: Using `console.log()`

```JavaScript
console.log("Hello, World!");
```

Output

```
Hello, World!
```

This is the most common way to display output in JavaScript.

---

2. Where Does the Output Appear?

In a Browser

If you run JavaScript inside a browser:

```HTML
<!DOCTYPE html>
<html>
<head>
    <title>Hello World</title>
</head>
<body>

<script>
    console.log("Hello, World!");
</script>

</body>
</html>
```

The output appears in the browser's **Developer Console**.

Open it using:

- Chrome: **F12** → Console
- Firefox: **F12** → Console
- Edge: **F12** → Console

In Node.js

Create a file:

```JavaScript
// app.js
console.log("Hello, World!");
```

Run:

```Bash
node app.js
```

Output:

```
Hello, World!
```

3. Displaying Output on a Web Page

Instead of printing to the console, JavaScript can display content directly on a webpage.

```HTML
<!DOCTYPE html>
<html>
<body>

<h1 id="demo"></h1>

<script>
document.getElementById("demo").innerHTML = "Hello, World!";
</script>

</body>
</html>
```

Output on webpage:

```
Hello, World!
```

4. Using `alert()`

JavaScript can also show a popup box.

```JavaScript
alert("Hello, World!");
```

Output:

A popup message appears with:

```
Hello, World!
```

5. Understanding the Syntax

```JavaScript
console.log("Hello, World!");
```

Breakdown

Part	Meaning
`console`	Browser's debugging object
`log()`	Method used to print output
`"Hello, World!"`	String (text)
`;`	Statement terminator

6. Why `console.log()` is Preferred

- Used for debugging
- Does not disturb users with popups
- Works in browsers and Node.js
- Most professional developers use it extensively

Example:

```JavaScript
console.log("Program Started");
console.log("Loading Data...");
console.log("Program Ended");
```

Interview Question

Q: What is the difference between `console.log()` and `alert()`?

console.log()	alert()
Prints to console	Shows popup
Used for debugging	Used for user notifications

console.log()	alert()
Does not stop execution	Pauses execution until closed

Exam-Oriented Definition

Hello World Program:
A Hello World program is the simplest JavaScript program used to display the message "Hello, World!" and verify that the JavaScript environment is working correctly.

```JavaScript
console.log("Hello, World!");
```

Key Takeaways

✅ JavaScript code can run in browsers and Node.js.
✅ `console.log()` is the standard way to print output.
✅ `alert()` displays popup messages.
✅ JavaScript can also modify webpage content using the DOM.

Practice

Write a program that prints:

```
My name is Krishna
I am learning JavaScript
```

using `console.log()`.

> Topic 2: Comments in JavaScript

Topic 2: Comments in JavaScript

Comments are lines in a program that are **ignored by the JavaScript engine**. They are used to explain code, add notes, temporarily disable code, and improve readability.

Why Do We Use Comments?

Imagine you write a program today and open it after 6 months.

Without comments:

```JavaScript
let r = 10;
let a = 3.14 * r * r;
```

You may forget what the code does.

With comments:

```JavaScript
// Calculate area of a circle

let r = 10;
let a = 3.14 * r * r;
```

Now the purpose is immediately clear.

Types of Comments in JavaScript

JavaScript supports two types of comments:

1. Single-Line Comments
2. Multi-Line Comments

---

1. Single-Line Comments

Single-line comments start with `//`.

Everything after `//` on that line is ignored.

Syntax

```JavaScript
// This is a comment
```

Example

```JavaScript
// Print a welcome message
console.log("Welcome to JavaScript");
```

Output

```
Welcome to JavaScript
```

The comment is not displayed.

Multiple Single-Line Comments

```JavaScript
// Program to calculate sum
// Created by Krishna
// Date: 2026

let a = 10;
```

```
let b = 20;

console.log(a + b);
```

Output:

```
30
```

2. Multi-Line Comments

Multi-line comments begin with `/*` and end with `*/`.

Syntax

```
JavaScript
/*
This is a
multi-line comment
*/
```

Example

```
JavaScript
/*
Program:
Find the area of a rectangle
*/

let length = 10;
let width = 5;

console.log(length * width);
```

Output:

```
50
```

Commenting Out Code

Developers often disable code temporarily during testing.

Example

```
JavaScript
console.log("Line 1");

// console.log("Line 2");

console.log("Line 3");
```

Output:

```
Line 1
Line 3
```

Since the second line is commented out, it does not execute.

Real-World Example

```JavaScript
// User information
let name = "Krishna";
let age = 21;

// Display information
console.log(name);
console.log(age);
```

Comments help other developers understand the code quickly.

Best Practices

Good Comment

```JavaScript
// Calculate total price including GST
let total = price + gst;
```

Bad Comment

```JavaScript
// Add x and y
let sum = x + y;
```

The code itself is already obvious. Comments should explain **why**, not repeat **what**.

Nested Comments

JavaScript does **not support nested multi-line comments**.

❌ Incorrect:

```JavaScript
/*
First comment

/*
Second comment
*/

*/
```

This causes errors.

Single-Line vs Multi-Line Comments

Feature	Single-Line (//)	Multi-Line (/* */)
Used for	Short notes	Long descriptions
Spans multiple lines	No	Yes
Easy for quick comments	Yes	No

Interview Questions

Q1: What are comments in JavaScript?

Comments are non-executable lines used to explain code and improve readability.

Q2: How many types of comments are there in JavaScript?

Two types:

- Single-line comments (`//`)
- Multi-line comments (`/* */`)

Q3: Does JavaScript execute comments?

No. Comments are ignored by the JavaScript engine.

Exam-Oriented Answer

Comments in JavaScript are statements that are ignored by the JavaScript engine and are used to explain code, improve readability, and assist debugging.

Types:

Single-Line Comment

```JavaScript
// This is a comment
```

Multi-Line Comment

```JavaScript
/************************
 This is a multi-line
 comment
************************/
```

Advantages

- Improves code readability

- Helps debugging
- Makes maintenance easier
- Documents program logic

Practice Questions

1. Write a program containing both single-line and multi-line comments.
2. Comment out one line of code and observe the output.
3. Create a program where comments explain each step of the logic.

Next Topic in JavaScript Fundamentals:

Variables in JavaScript (var, let, const) — one of the most important topics in JavaScript.

> Topic 3: Variables ('let', 'var', 'const')

Topic 3: Variables in JavaScript (`var`, `let`, `const`)

Variables are used to **store data in memory** so that it can be used later in a program.

Think of a variable as a **labeled box**.

📦 **`name`** → stores **`"Krishna"`**
📦 **`age`** → stores **`21`**

Whenever you need the value, you use the variable's name.

What is a Variable?

A variable is a named storage location used to hold data.

Example

```
JavaScript
let name = "Krishna";

console.log(name);
```

Output

```
Krishna
```

Why Do We Need Variables?

Without variables:

```JavaScript
console.log("Krishna");
console.log("Krishna");
console.log("Krishna");
```

With variables:

```JavaScript
let name = "Krishna";

console.log(name);
console.log(name);
console.log(name);
```

If the value changes, you only update it once.

Variable Declaration Syntax

```JavaScript
keyword variableName = value;
```

Example:

```JavaScript
let age = 21;
```

Here:

- `let` → keyword
- `age` → variable name
- `21` → value

JavaScript Variable Keywords

JavaScript provides three ways to declare variables:

1. `var` (old way)
2. `let` (modern way)
3. `const` (constant values)

---

1. `var`

Before ES6 (2015), `var` was the only way to declare variables.

Example

```JavaScript
```

```
var name = "Krishna";

console.log(name);
```

Output:

```
Krishna
```

Re-declaration Allowed

```
JavaScript
var city = "Meerut";
var city = "Delhi";

console.log(city);
```

Output:

```
Delhi
```

No error occurs.

Reassignment Allowed

```
JavaScript
var age = 20;

age = 21;

console.log(age);
```

Output:

```
21
```

Problems with `var`

`var` is function-scoped, which can create unexpected behavior.

```
JavaScript
if (true) {
    var x = 10;
}

console.log(x);
```

Output:

```
10
```

Even though **`x`** was declared inside the block, it is accessible outside.

This often leads to bugs.

2. `let`

Introduced in ES6 (2015).

`let` is the preferred way to declare variables whose values may change.

Example

```
JavaScript
let age = 21;

console.log(age);
```

Reassignment Allowed

```
JavaScript
let age = 21;

age = 22;

console.log(age);
```

Output:

```
22
```

Re-declaration Not Allowed

```
JavaScript
let age = 21;
let age = 22;
```

Output:

```
SyntaxError
```

Block Scope

```
JavaScript
if (true) {
    let x = 10;
}

console.log(x);
```

Output:

```
ReferenceError
```

`x` exists only inside the block.

This makes code safer.

---

3. `const`

`const` is used for values that should not change.

Example

```JavaScript
const PI = 3.14159;

console.log(PI);
```

Output:

```
3.14159
```

Reassignment Not Allowed

```JavaScript
const PI = 3.14;

PI = 3.14159;
```

Output:

```
TypeError
```

Declaration Without Value Not Allowed

❌ Incorrect

```JavaScript
const age;
```

Output:

```
SyntaxError
```

✅ Correct

```JavaScript
const age = 21;
```

Block Scope

```JavaScript
if (true) {
    const x = 100;
}

console.log(x);
```

Output:

```
ReferenceError
```

Important Note About `const`

Many beginners think **`const`** makes everything immutable.

Not true.

For objects and arrays, the reference cannot change, but contents can.

Example

```JavaScript
const student = {
    name: "Krishna"
};

student.name = "Rahul";

console.log(student.name);
```

Output:

```
Rahul
```

Allowed because only the object's property changed.

Variable Naming Rules

Valid Names

```JavaScript
let age = 21;
let firstName = "Krishna";
let _count = 10;
let $price = 100;
```

Invalid Names

```JavaScript
```

```
let 1age = 21;
let my-name = "Krishna";
let let = 10;
```

Reasons:

- Cannot start with a number
- Hyphen (-) is not allowed
- Reserved keywords cannot be used

Naming Conventions

JavaScript uses **camelCase**.

✅ Good

```JavaScript
let firstName;
let totalPrice;
let studentAge;
```

❌ Avoid

```JavaScript
let firstname;
let TOTALPRICE;
let student_age;
```

Difference Between `var`, `let`, and `const`

Feature	var	let	const
Re-declaration	✅ Yes	❌ No	❌ No
Reassignment	✅ Yes	✅ Yes	❌ No
Block Scope	❌ No	✅ Yes	✅ Yes
Function Scope	✅ Yes	❌ No	❌ No
Modern Usage	❌ Rare	✅ Common	✅ Common

Which One Should You Use?

Modern JavaScript practice:

1. Use **`const`** by default.
2. Use **`let`** if the value will change.
3. Avoid **`var`**.

Example:

```JavaScript
const PI = 3.14159;
let score = 0;

score++;

console.log(score);
```

Real-World Example

```JavaScript
const company = "Google";
let salary = 50000;

salary = 60000;

console.log(company);
console.log(salary);
```

Output:

```
Google
60000
```

Interview Questions

Q1: Difference between `let` and `var`?

- **`var`** is function-scoped.
- **`let`** is block-scoped.
- **`var`** allows re-declaration.
- **`let`** does not.

Q2: Can a `const` variable be changed?

No, it cannot be reassigned.

```JavaScript
const PI = 3.14;
PI = 3.14159; // Error
```

Q3: Which variable type should be preferred?

Use:

- **`const`** by default
- **`let`** when value changes
- Avoid **`var`**

Exam-Oriented Answer

A **variable** is a named memory location used to store data in a JavaScript program.

JavaScript provides three keywords:

- `var` → function-scoped, allows re-declaration.
- `let` → block-scoped, allows reassignment.
- `const` → block-scoped, does not allow reassignment.

Example:

```JavaScript
const PI = 3.14;
let age = 21;
var name = "Krishna";
```

Practice Questions

Question 1

Predict the output:

```JavaScript
let x = 10;
x = 20;

console.log(x);
```

Answer: `20`

Question 2

Predict the output:

```JavaScript
const PI = 3.14;
PI = 3.14159;
```

Answer: Error

Question 3

Predict the output:

```JavaScript
if (true) {
    let a = 10;
}
```

```
console.log(a);
```

Answer: `ReferenceError`

Key Takeaway

✅ `let` and `const` are modern JavaScript variables.
✅ `let` = value can change.
✅ `const` = value cannot be reassigned.
✅ Avoid `var` in modern development.

Next Topic: Data Types in JavaScript (Primitive & Non-Primitive Types).

> Topic 4: Primitive data types

Topic 4: Primitive Data Types in JavaScript

Before understanding primitive data types, let's first understand what a **data type** is.

What is a Data Type?

A data type specifies the kind of value a variable can store.

Examples:

- `25` → Number
- `"Krishna"` → String
- `true` → Boolean

```
JavaScript
let age = 21;
let name = "Krishna";
let isStudent = true;
```

Here, each variable stores a different type of data.

Types of Data in JavaScript

JavaScript data types are divided into two categories:

1. Primitive Data Types
2. Non-Primitive (Reference) Data Types

In this topic, we'll focus on **Primitive Data Types**.

What are Primitive Data Types?

Primitive data types are the most basic data types in JavaScript.

They store a **single value directly** and are **immutable** (cannot be changed in place).

JavaScript has **7 Primitive Data Types**:

1. String
2. Number
3. Boolean
4. Undefined
5. Null
6. BigInt
7. Symbol

---

1. String

A string represents textual data.

Strings can be written using:

- Single Quotes ' '
- Double Quotes " "
- Backticks ` `

Example

```JavaScript
let name = "Krishna";
let city = 'Meerut';
let course = `JavaScript`;

console.log(name);
console.log(city);
console.log(course);
```

Output

```
Krishna
Meerut
JavaScript
```

String Properties

```JavaScript
```

```
let language = "JavaScript";

console.log(language.length);
```

Output:

```
10
```

2. Number

Represents both integers and decimal numbers.

Example

```
JavaScript
let age = 21;
let salary = 50000.75;

console.log(age);
console.log(salary);
```

Output:

```
21
50000.75
```

Special Number Values

Infinity

```
JavaScript
console.log(10 / 0);
```

Output:

```
Infinity
```

NaN (Not a Number)

```
JavaScript
console.log("Hello" * 5);
```

Output:

```
NaN
```

3. Boolean

Represents only two values:

- `true`
- `false`

Example

```
JavaScript
let isLoggedIn = true;
let hasPermission = false;

console.log(isLoggedIn);
console.log(hasPermission);
```

Output:

```
true
false
```

Real-World Example

```
JavaScript
let isStudent = true;

if (isStudent) {
    console.log("Discount Applied");
}
```

Output:

```
Discount Applied
```

4. Undefined

When a variable is declared but no value is assigned, it becomes **`undefined`**.

Example

```
JavaScript
let age;

console.log(age);
```

Output:

```
undefined
```

Why Does Undefined Occur?

Because memory is allocated but no value is stored.

```
JavaScript
```

```
let name;

console.log(typeof name);
```

Output:

```
undefined
```

5. Null

`null` represents an intentional absence of value.

Example

```
JavaScript
let user = null;

console.log(user);
```

Output:

```
null
```

Difference Between Undefined and Null

Undefined	Null
Value not assigned	Intentionally empty
Assigned automatically	Assigned manually
Type is undefined	Type is object (historical bug)

Example:

```
JavaScript
let a;
let b = null;

console.log(a);
console.log(b);
```

Output:

```
undefined
null
```

6. BigInt

Used for very large integers beyond the safe Number limit.

Example

```JavaScript
let bigNumber = 1234567890123456789012345678890n;

console.log(bigNumber);
```

Output:

```
1234567890123456789012345678890n
```

Notice the `n` at the end.

Why BigInt?

JavaScript Number can safely store only up to:

```JavaScript
console.log(Number.MAX_SAFE_INTEGER);
```

Output:

```
9007199254740991
```

For larger numbers, use BigInt.

---

7. Symbol

A Symbol creates a unique identifier.

Example

```JavaScript
let id1 = Symbol("id");
let id2 = Symbol("id");

console.log(id1 === id2);
```

Output:

```
false
```

Even though descriptions are the same, each Symbol is unique.

Use Case of Symbol

Used to create unique object properties.

```JavaScript
```

```
const userId = Symbol("userId");
```

Common in advanced JavaScript development.

Checking Data Types with `typeof`

JavaScript provides the **`typeof`** operator.

Example

```
JavaScript
console.log(typeof "Krishna");
console.log(typeof 21);
console.log(typeof true);
```

Output:

```
string
number
boolean
```

More Examples

```
JavaScript
console.log(typeof undefined);
console.log(typeof null);
console.log(typeof 100n);
console.log(typeof Symbol("id"));
```

Output:

```
undefined
object
bigint
symbol
```

Complete Example

```
JavaScript
let name = "Krishna";
let age = 21;
let isStudent = true;
let city;
let salary = null;
let bigNum = 12345678901234567890n;
let id = Symbol("ID");

console.log(typeof name);
console.log(typeof age);
console.log(typeof isStudent);
console.log(typeof city);
console.log(typeof salary);
console.log(typeof bigNum);
console.log(typeof id);
```

Output:

```
string
number
boolean
undefined
object
bigint
symbol
```

Primitive Data Types Summary

Data Type	Example
String	`"Krishna"`
Number	`21`
Boolean	`true`
Undefined	`let x;`
Null	`null`
BigInt	`123456789n`
Symbol	`Symbol("id")`

Interview Questions

Q1: How many primitive data types are there in JavaScript?

There are **7 primitive data types**:

- String
- Number
- Boolean
- Undefined
- Null
- BigInt
- Symbol

Q2: What is the difference between `null` and `undefined`?

- **`undefined`** → Variable declared but not assigned.
- **`null`** → Intentional absence of value.

Q3: What is the output?

```
JavaScript
```

```
console.log(typeof null);
```

Output:

```
object
```

This is a historical bug in JavaScript.

Q4: What is BigInt?

BigInt is a primitive data type used to store integers larger than `Number.MAX_SAFE_INTEGER`.

Exam-Oriented Answer

Primitive Data Types are basic built-in data types that store a single value directly.

JavaScript has seven primitive data types:

1. String
2. Number
3. Boolean
4. Undefined
5. Null
6. BigInt
7. Symbol

Example:

```JavaScript
let name = "Krishna";
let age = 21;
let isStudent = true;
```

Primitive values are immutable and stored directly in memory.

Practice Questions

Predict the Output

```JavaScript
let x;
console.log(typeof x);
```

Answer:

```
undefined
```

Predict the Output

```JavaScript
console.log(typeof 100n);
```

Answer:

```
bigint
```

Predict the Output

```JavaScript
console.log(typeof Symbol("id"));
```

Answer:

```
symbol
```

Key Takeaways

✅ JavaScript has **7 primitive data types**.
✅ `typeof` is used to check a variable's type.
✅ `undefined` and `null` are different concepts.
✅ `BigInt` handles very large integers.
✅ `Symbol` creates unique identifiers.

Next Topic:

Non-Primitive (Reference) Data Types: Object, Array, Function, Date, etc.

> What does it mean primitive data types are immutable?

When we say **primitive data types are immutable**, it means:

> **Once a primitive value is created, its actual value cannot be changed.**
> If you appear to "change" it, JavaScript actually creates a **new value** and assigns it to the variable.

Example 1: String

```JavaScript
let name = "Krishna";

name = "Rahul";

console.log(name);
```

Output:

```
Rahul
```

It looks like `"Krishna"` was changed to `"Rahul"`.

But internally:

1. `"Krishna"` remains unchanged.
2. JavaScript creates a new string `"Rahul"`.
3. The variable `name` now points to the new string.

```
Before:
name → "Krishna"

After:
name → "Rahul"
```

The original string was never modified.

Example 2: Trying to Modify a String Character

```JavaScript
let str = "Hello";

str[0] = "Y";

console.log(str);
```

Output:

```
Hello
```

Why?

Because strings are immutable.

JavaScript does not allow changing individual characters inside a string.

Example 3: Number

```JavaScript
let age = 21;

age = age + 1;

console.log(age);
```

Output:

```
22
```

Internally:

1. `21` is not modified.
2. JavaScript creates a new value `22`.
3. The variable `age` now stores `22`.

Example 4: Boolean

```JavaScript
let isLoggedIn = true;

isLoggedIn = false;
```

Again:

- `true` remains unchanged.
- `false` is a new value.
- The variable now refers to the new value.

Why Is This Important?

Because primitive values are copied by value.

```JavaScript
let a = 10;
let b = a;

b = 20;

console.log(a);
console.log(b);
```

Output:

```
10
20
```

Changing `b` does not affect `a`.

Each variable has its own copy of the primitive value.

Primitive vs Object (Mutable)

Primitive:

```JavaScript
let x = "Hello";
x = "World";
```

A new string is created.

Object:

```JavaScript
let user = {
    name: "Krishna"
};

user.name = "Rahul";

console.log(user.name);
```

Output:

```
Rahul
```

Objects are **mutable**, meaning their contents can be changed after creation.

Easy Exam Definition

Immutable means a value cannot be modified after it is created. Any apparent modification results in the creation of a new value rather than changing the existing one.

Examples of immutable primitive types:

- String
- Number
- Boolean
- Undefined
- Null
- BigInt
- Symbol

A common interview question is:

Q: Are strings mutable in JavaScript?

Answer: No. Strings are immutable. Individual characters cannot be changed after the string is created.

```JavaScript
let str = "Hello";
str[0] = "Y";

console.log(str); // Hello
```

Topic 5: Operators (Arithmetic, Relational, logical and others)

Topic 5: Operators in JavaScript

What is an Operator?

An **operator** is a symbol that performs an operation on one or more operands (values or variables).

Example

```JavaScript
let a = 10;
let b = 5;

console.log(a + b);
```

Output:

```
15
```

Here:

- `a` and `b` are operands.
- `+` is the operator.

Types of Operators in JavaScript

1. Arithmetic Operators
2. Assignment Operators
3. Relational (Comparison) Operators
4. Logical Operators
5. Increment & Decrement Operators
6. Ternary Operator
7. Type Operators
8. Nullish Coalescing Operator
9. Optional Chaining Operator

---

1. Arithmetic Operators

Used to perform mathematical calculations.

Operator	Meaning	Example
`+`	Addition	`10 + 5`
`-`	Subtraction	`10 - 5`
`*`	Multiplication	`10 * 5`
`/`	Division	`10 / 5`
`%`	Modulus (Remainder)	`10 % 3`

Operator	Meaning	Example
**	Exponentiation	2 ** 3

Addition

```JavaScript
let a = 10;
let b = 20;

console.log(a + b);
```

Output:

```
30
```

Subtraction

```JavaScript
console.log(20 - 10);
```

Output:

```
10
```

Multiplication

```JavaScript
console.log(5 * 4);
```

Output:

```
20
```

Division

```JavaScript
console.log(20 / 5);
```

Output:

```
4
```

Modulus (%)

Returns the remainder.

```
JavaScript
console.log(10 % 3);
```

Output:

```
1
```

Exponentiation (**)

```
JavaScript
console.log(2 ** 4);
```

Output:

```
16
```

2. Assignment Operators

Used to assign values to variables.

Basic Assignment

```
JavaScript
let x = 10;
```

Compound Assignment

Operator	Example	Equivalent
+=	x += 5	x = x + 5
-=	x -= 5	x = x - 5
*=	x *= 5	x = x * 5
/=	x /= 5	x = x / 5
%=	x %= 5	x = x % 5

Example:

```
JavaScript
let x = 10;

x += 5;
```

```
console.log(x);
```

Output:

```
15
```

3. Relational (Comparison) Operators

Used to compare values.

The result is always:

- `true`
- `false`

Operator	Meaning
`==`	Equal
`===`	Strict Equal
`!=`	Not Equal
`!==`	Strict Not Equal
`>`	Greater Than
`<`	Less Than
`>=`	Greater Than or Equal
`<=`	Less Than or Equal

Equal (==)

Checks value only.

```
JavaScript
console.log(5 == "5");
```

Output:

```
true
```

JavaScript converts `"5"` to number.

Strict Equal (===)

Checks value and type.

```JavaScript
console.log(5 === "5");
```

Output:

```
false
```

Because:

```
5      → number
"5"    → string
```

Greater Than

```JavaScript
console.log(10 > 5);
```

Output:

```
true
```

Less Than

```JavaScript
console.log(10 < 5);
```

Output:

```
false
```

Important Interview Point

Always prefer:

```JavaScript
===
```

instead of

```JavaScript
==
```

because it avoids unexpected type conversion.

---

4. Logical Operators

Used to combine or invert conditions.

Operator	Meaning
&&	AND
`	
!	NOT

AND (&&)

Returns **true** only if both conditions are true.

```JavaScript
let age = 20;
let hasLicense = true;

console.log(age >= 18 && hasLicense);
```

Output:

```
true
```

Truth Table

A	B	A && B
T	T	T
T	F	F
F	T	F
F	F	F

OR (||)

Returns **true** if at least one condition is true.

```JavaScript
console.log(true || false);
```

Output:

```
true
```

Truth Table

A	B	A || B
T	T	T
T	F	T
F	T	T
F	F	F

NOT (!)

Reverses a boolean value.

```
JavaScript
console.log(!true);
```

Output:

```
false
```

5. Increment and Decrement Operators

Increment (++)

Adds 1.

```
JavaScript
let x = 5;

x++;

console.log(x);
```

Output:

```
6
```

Decrement (--)

Subtracts 1.

```
JavaScript
let x = 5;

x--;

console.log(x);
```

Output:

```
4
```

Prefix vs Postfix

Prefix

```
JavaScript
let x = 5;

console.log(++x);
```

Output:

```
6
```

Increment first, then use.

Postfix

```
JavaScript
let x = 5;

console.log(x++);
```

Output:

```
5
```

Then `x` becomes `6`.

---

6. Ternary Operator

Shortcut for `if-else`.

Syntax

```
JavaScript
condition ? value1 : value2
```

Example:

```
JavaScript
let age = 20;

let result = age >= 18 ? "Adult" : "Minor";

console.log(result);
```

Output:

```
Adult
```

7. Type Operators

typeof

Used to find data type.

```JavaScript
console.log(typeof "Krishna");
```

Output:

```
string
```

instanceof

Checks object type.

```JavaScript
let arr = [];

console.log(arr instanceof Array);
```

Output:

```
true
```

8. Nullish Coalescing Operator (??)

Returns the right value only when the left value is:

- `null`
- `undefined`

Example:

```JavaScript
let name = null;

console.log(name ?? "Guest");
```

Output:

```
Guest
```

9. Optional Chaining (?.)

Safely accesses nested properties.

Without Optional Chaining:

```
JavaScript
let user = null;

console.log(user.name);
```

Output:

```
Error
```

With Optional Chaining:

```
JavaScript
let user = null;

console.log(user?.name);
```

Output:

```
undefined
```

No error occurs.

Operator Precedence

Example:

```
JavaScript
console.log(10 + 5 * 2);
```

Output:

```
20
```

Because multiplication happens before addition.

```
5 * 2 = 10
10 + 10 = 20
```

Use parentheses to control precedence:

```
JavaScript
console.log((10 + 5) * 2);
```

Output:

```
30
```

Summary Table

Category	Operators
Arithmetic	`+ - * / % **`
Assignment	`= += -= *= /= %=`
Comparison	`== === != !== > < >= <=`
Logical	`&&
Increment/Decrement	`++ --`
Ternary	`?:`
Type	`typeof instanceof`
Nullish	`??`
Optional Chaining	`?.`

Interview Questions

Q1: Difference between == and ===?

```JavaScript
5 == "5"   // true
5 === "5"  // false
```

- == checks value only.
- === checks value and type.

Q2: Difference between && and ||?

- && → all conditions must be true.
- || → at least one condition must be true.

Q3: Output?

```JavaScript
let x = 5;
console.log(x++);
```

Output:

```
5
```

Then x becomes 6.

Exam-Oriented Definition

Operators are special symbols used to perform operations on operands. JavaScript provides arithmetic, assignment, comparison, logical, ternary, type, nullish coalescing, and optional chaining operators to manipulate and evaluate data.

Practice Questions

Predict the output:

```JavaScript
console.log(10 % 4);
```

Answer: `2`

```JavaScript
console.log(5 === "5");
```

Answer: `false`

```JavaScript
console.log(!(10 > 5));
```

Answer: `false`

```JavaScript
let x = 10;
x += 5;
console.log(x);
```

Answer: `15`

Next Topic:

Type Conversion & Type Coercion (Implicit vs Explicit Conversion) — one of the most important and frequently asked JavaScript concepts.

> Topic 6: Type coercion

Topic 6: Type Coercion in JavaScript

Type Coercion is one of JavaScript's most important and sometimes confusing concepts.

What is Type Coercion?

Type Coercion is the automatic conversion of one data type into another by JavaScript when performing operations.

In simple words:

JavaScript tries to convert values automatically so an operation can be completed.

Example 1

```JavaScript
console.log("5" + 2);
```

Output:

```
52
```

Why?

- `"5"` is a string.
- `2` is a number.
- JavaScript converts `2` into `"2"`.
- Then concatenates them.

```JavaScript
"5" + "2" = "52"
```

This automatic conversion is called **Type Coercion**.

Why Does JavaScript Perform Type Coercion?

JavaScript is a **dynamically typed language**.

Variables do not have fixed data types.

```JavaScript
let value = 10;

value = "Hello";
```

Both are valid.

Because of this flexibility, JavaScript often converts types automatically.

Types of Conversion

There are two types:

1. Implicit Type Coercion (Automatic)
2. Explicit Type Conversion (Manual)

In this topic, we focus mainly on **Implicit Type Coercion**.

String Coercion

When a string is involved with the + operator, JavaScript usually converts everything to a string.

Example

```JavaScript
console.log("10" + 5);
```

Output:

```
105
```

Internally:

```JavaScript
"10" + "5"
```

More Examples

```JavaScript
console.log("Hello " + "World");
```

Output:

```
Hello World
```

```JavaScript
console.log("Age: " + 21);
```

Output:

```
Age: 21
```

Number becomes a string.

Numeric Coercion

Operators like:

```JavaScript
-
*
/
%
```

usually convert strings to numbers.

Example

```
JavaScript
console.log("10" - 5);
```

Output:

```
5
```

Internally:

```
JavaScript
10 - 5
```

Multiplication

```
JavaScript
console.log("10" * 2);
```

Output:

```
20
```

Division

```
JavaScript
console.log("20" / 2);
```

Output:

```
10
```

Modulus

```
JavaScript
console.log("10" % 3);
```

Output:

```
1
```

Interesting Example

```
JavaScript
```

```
console.log("10" + 5);
```

Output:

```
105
```

But

```
JavaScript
console.log("10" - 5);
```

Output:

```
5
```

Why?

Because:

- `+` prefers string concatenation.
- `-` only works mathematically.

Boolean Coercion

JavaScript converts values to Boolean in conditions.

```
JavaScript
if ("Hello") {
    console.log("Executed");
}
```

Output:

```
Executed
```

Because non-empty strings are considered `true`.

Truthy and Falsy Values

Falsy Values

These become `false` when converted to Boolean:

```
JavaScript
false
0
-0
0n
""
null
```

```
undefined
NaN
```

Example

```JavaScript
if (0) {
    console.log("Run");
}
```

Nothing prints because `0` is falsy.

Truthy Values

Everything else is truthy.

Examples:

```JavaScript
"Hello"
"0"
[]
{}
100
-50
true
```

Example

```JavaScript
if ("0") {
    console.log("Truthy");
}
```

Output:

```
Truthy
```

Even though it contains 0, it is a non-empty string.

Equality Coercion (==)

The == operator performs automatic type conversion.

Example

```JavaScript
console.log(5 == "5");
```

Output:

```
true
```

Internally:

```JavaScript
5 == 5
```

More Examples

```JavaScript
console.log(true == 1);
```

Output:

```
true
```

Because:

```JavaScript
true → 1
```

```JavaScript
console.log(false == 0);
```

Output:

```
true
```

Because:

```JavaScript
false → 0
```

Strange Examples

These are famous JavaScript interview questions.

Example 1

```JavaScript
console.log("" == 0);
```

Output:

```
true
```

Example 2

```JavaScript
console.log(null == undefined);
```

Output:

```
true
```

Example 3

```JavaScript
console.log([] == "");
```

Output:

```
true
```

These happen because of complex coercion rules.

Use === Instead of ==

```JavaScript
console.log(5 === "5");
```

Output:

```
false
```

No type conversion occurs.

=== checks:

1. Value
2. Data Type

Best Practice

Always prefer:

```JavaScript
===
```

instead of:

```JavaScript
==
```

unless you specifically need coercion.

Type Coercion with Boolean Operators

AND (&&)

```
JavaScript
console.log("Hello" && 100);
```

Output:

```
100
```

OR (||)

```
JavaScript
console.log("" || "Default");
```

Output:

```
Default
```

Because "" is falsy.

Real-World Example

```
JavaScript
let username = "";

let displayName = username || "Guest";

console.log(displayName);
```

Output:

```
Guest
```

Used frequently in applications.

Common Interview Outputs

Output 1

```
JavaScript
console.log("5" + 2);
```

Output:

```
52
```

Output 2

```JavaScript
console.log("5" - 2);
```

Output:

```
3
```

Output 3

```JavaScript
console.log(true + 1);
```

Output:

```
2
```

Because:

```JavaScript
true → 1
```

Output 4

```JavaScript
console.log(false + 5);
```

Output:

```
5
```

Because:

```JavaScript
false → 0
```

Output 5

```JavaScript
console.log(null + 1);
```

Output:

```
1
```

Because:

```JavaScript
null → 0
```

Type Coercion vs Type Conversion

Type Coercion	Type Conversion
Automatic	Manual
Done by JavaScript	Done by programmer
Implicit	Explicit

Example:

```JavaScript
"5" + 2
```

Coercion

Example:

```JavaScript
Number("5")
```

Conversion

Exam-Oriented Definition

Type Coercion is the automatic conversion of one data type into another by JavaScript during expression evaluation or comparison.

Example:

```JavaScript
console.log("5" + 2);
```

Output:

```
52
```

JavaScript automatically converts `2` to `"2"` and performs string concatenation.

Summary

String Coercion

```JavaScript
"5" + 2
```

Result:

```
52
```

Numeric Coercion

```JavaScript
"5" - 2
```

Result:

```
3
```

Boolean Coercion

```JavaScript
if ("Hello")
```

Evaluates to:

```
true
```

Equality Coercion

```JavaScript
5 == "5"
```

Result:

```
true
```

Strict Equality

```JavaScript
5 === "5"
```

Result:

```
false
```

Practice Questions

Predict the output:

```JavaScript
console.log("10" + 5);
```

Answer: `"105"`

```JavaScript
console.log("10" - 5);
```

Answer: `5`

```JavaScript
console.log(true + 2);
```

Answer: `3`

```JavaScript
console.log(false == 0);
```

Answer: `true`

Key Takeaway

✅ JavaScript automatically converts types when needed.
✅ `+` often performs string concatenation.
✅ `-`, `*`, `/`, `%` usually convert strings to numbers.
✅ Understand **truthy** and **falsy** values.
✅ Prefer `===` over `==` to avoid unexpected coercion.

Next Topic:

Template Literals and String Interpolation — a modern ES6 feature used extensively in real-world JavaScript development.

> Topic 7: Does it mean implicit type conversion is type coercion and also explain explicit type conversion

Yes, exactly.

Relationship Between Type Coercion and Type Conversion

1. Implicit Type Conversion = Type Coercion

When JavaScript automatically converts one data type to another without your instruction, it is called **Type Coercion** (or Implicit Type Conversion).

Example:

```JavaScript
console.log("5" + 2);
```

Output:

```
52
```

What happened?

```
JavaScript
"5" + 2
↓
"5" + "2"
↓
"52"
```

JavaScript automatically converted `2` into `"2"`.

This is **Implicit Type Conversion (Type Coercion)**.

---

2. Explicit Type Conversion

When **you intentionally convert** one data type into another using built-in functions, it is called **Explicit Type Conversion** (or Type Casting).

Example:

```
JavaScript
console.log(Number("5") + 2);
```

Output:

```
7
```

What happened?

```
JavaScript
"5"
↓
Number("5")
↓
5
```

You explicitly instructed JavaScript to convert the string into a number.

Comparison

Feature	Implicit Conversion (Type Coercion)	Explicit Conversion
Who converts?	JavaScript	Programmer
Automatic?	Yes	No
Control	Less	Full
Preferred?	Sometimes confusing	Recommended

Explicit Type Conversion

JavaScript provides several conversion functions:

1. `String()`
2. `Number()`
3. `Boolean()`
4. `parseInt()`
5. `parseFloat()`

---

1. Converting to String

Using `String()`

```JavaScript
let age = 21;

let strAge = String(age);

console.log(strAge);
console.log(typeof strAge);
```

Output:

```
21
string
```

Another Way

```JavaScript
let num = 100;

console.log(num.toString());
```

Output:

```
100
```

2. Converting to Number

Using `Number()`

```JavaScript
let value = "123";

let num = Number(value);

console.log(num);
console.log(typeof num);
```

Output:

```
123
number
```

Invalid Conversion

```JavaScript
console.log(Number("Hello"));
```

Output:

```
NaN
```

Because `"Hello"` cannot be converted into a valid number.

Boolean to Number

```JavaScript
console.log(Number(true));
console.log(Number(false));
```

Output:

```
1
0
```

Conversion rules:

```
true  → 1
false → 0
```

3. Converting to Boolean

Using `Boolean()`

```JavaScript
console.log(Boolean(1));
console.log(Boolean(0));
```

Output:

```
true
false
```

Common Boolean Conversions

Value	Result
`1`	`true`
`0`	`false`
`"Hello"`	`true`
`""`	`false`
`null`	`false`
`undefined`	`false`

Example:

```JavaScript
console.log(Boolean("JavaScript"));
console.log(Boolean(""));
```

Output:

```
true
false
```

4. parseInt()

Used to extract an integer from a string.

```JavaScript
console.log(parseInt("100"));
```

Output:

```
100
```

With Mixed Content

```JavaScript
console.log(parseInt("100px"));
```

Output:

```
100
```

It stops reading when it encounters a non-numeric character.

Another Example

```JavaScript
console.log(parseInt("123abc456"));
```

Output:

```
123
```

5. parseFloat()

Used for decimal numbers.

```JavaScript
console.log(parseFloat("99.99"));
```

Output:

```
99.99
```

Mixed Content

```JavaScript
console.log(parseFloat("99.99kg"));
```

Output:

```
99.99
```

Number() vs parseInt()

```JavaScript
console.log(Number("100px"));
```

Output:

```
NaN
```

But:

```JavaScript
console.log(parseInt("100px"));
```

Output:

```
100
```

Difference

Function	Behavior
`Number()`	Entire string must be numeric
`parseInt()`	Reads until invalid character

Real-World Example

Suppose a user enters age through a form:

```JavaScript
let age = "21";
```

Age comes as a string.

If you do:

```JavaScript
console.log(age + 5);
```

Output:

```
215
```

Wrong.

Correct approach:

```JavaScript
let age = "21";

console.log(Number(age) + 5);
```

Output:

```
26
```

This is one of the most common uses of explicit conversion.

Interview Questions

Q1: Is Type Coercion the same as Implicit Type Conversion?

Yes.

```
Type Coercion = Implicit Type Conversion
```

Both refer to automatic conversion by JavaScript.

Q2: What is Explicit Type Conversion?

It is the manual conversion of one data type into another using functions like:

```JavaScript
Number()
String()
Boolean()
parseInt()
parseFloat()
```

Q3: Output?

```JavaScript
console.log(Number("50") + 10);
```

Output:

```
60
```

Q4: Output?

```JavaScript
console.log(parseInt("50px"));
```

Output:

```
50
```

Memory Trick

Think of it like this:

```
Implicit Conversion
=
JavaScript decides

Explicit Conversion
=
You decide
```

or

```
Type Coercion
=
Automatic

Type Conversion
=
Manual
```

Although technically "type conversion" is a broad term that includes both implicit and explicit conversion, in interviews and day-to-day discussions:

- **Type Coercion** usually means **implicit conversion**.
- **Type Conversion/Casting** usually means **explicit conversion**.

Quick Revision Table

Expression	Result	Type
`"5" + 2`	`"52"`	Coercion
`"5" - 2`	`3`	Coercion
`Number("5")`	`5`	Explicit
`String(10)`	`"10"`	Explicit
`Boolean(1)`	`true`	Explicit
`parseInt("100px")`	`100`	Explicit

Key Takeaway

✅ Implicit Type Conversion = Type Coercion (automatic).
✅ Explicit Type Conversion = Manual conversion by the programmer.
✅ Use `Number()`, `String()`, and `Boolean()` frequently in real-world JavaScript.
✅ Prefer explicit conversion when you want predictable behavior.

> Topic 8: Introduction to Memory (Stack vs Heap - Basic idea)

Topic 8: Introduction to Memory (Stack vs Heap - Basic Idea)

This topic is extremely important because it helps you understand:

- How variables are stored in memory
- Why primitives behave differently from objects
- Why copying an object behaves differently from copying a number or string

For now, we'll learn the **basic idea**. Later, when you study Execution Contexts and Memory Management, you'll understand it in greater depth.

What is Memory?

When a JavaScript program runs, the computer allocates memory to store:

- Variables
- Functions
- Objects
- Arrays
- Data

JavaScript mainly uses two memory areas:

```
Memory
├── Stack Memory
└── Heap Memory
```

Stack Memory

Stack Memory stores:

✅ Primitive Data Types

- Number
- String
- Boolean
- Null
- Undefined
- BigInt
- Symbol

Stack memory is:

- Fast
- Small
- Organized
- Stores actual values directly

Example

```JavaScript
let age = 21;
let name = "Krishna";
```

Memory:

```
Stack

age  → 21
name → "Krishna"
```

The actual values are stored directly.

Heap Memory

Heap Memory stores:

✅ Non-Primitive (Reference) Data Types

- Objects
- Arrays
- Functions

Heap memory is:

- Larger
- More flexible
- Slightly slower
- Stores actual objects

Example

```JavaScript
let student = {
    name: "Krishna",
    age: 21
};
```

Memory:

```
Stack                  Heap

student ──────────► { name: "Krishna", age: 21 }
```

The object itself is stored in Heap.

The variable stores only a reference (address).

Primitive Example (Stack)

```JavaScript
let a = 10;
let b = a;

b = 20;

console.log(a);
console.log(b);
```

Output:

```
10
20
```

How Memory Works

Initially:

```
Stack

a → 10
```

After:

```
JavaScript
let b = a;
```

```
Stack

a → 10
b → 10
```

A copy of the value is created.

After:

```
JavaScript
b = 20;
```

```
Stack

a → 10
b → 20
```

Changing `b` does not affect `a`.

This is called:

> Pass by Value

Object Example (Heap)

```
JavaScript
let user1 = {
    name: "Krishna"
};

let user2 = user1;

user2.name = "Rahul";

console.log(user1.name);
console.log(user2.name);
```

Output:

```
Rahul
Rahul
```

Many beginners expect:

```
Krishna
Rahul
```

But that's not what happens.

What Happened?

Step 1

```JavaScript
let user1 = {
    name: "Krishna"
};
```

Memory:

```
Stack                Heap

user1 ──────►   { name: "Krishna" }
```

Step 2

```JavaScript
let user2 = user1;
```

Memory:

```
Stack                    Heap

user1 ──────┐
            ▼
        { name: "Krishna" }
            ▲
user2 ──────┘
```

Both variables point to the same object.

Step 3

```JavaScript
user2.name = "Rahul";
```

Memory:

```
Stack                    Heap

user1 ──────┐
            ▼
        { name: "Rahul" }
            ▲
user2 ──────┘
```

Only one object exists.

Both variables refer to it.

Primitive vs Reference Types

Primitive	Reference
Stored in Stack	Stored in Heap
Actual value stored	Address stored
Copy creates new value	Copy creates new reference
Independent copies	Shared object
Fast access	Slightly slower

Real-World Analogy

Imagine:

Primitive

```
a = 100
b = a
```

Like photocopying a document.

Now you have:

```
Copy 1 → 100
Copy 2 → 100
```

Changing one copy doesn't affect the other.

Object

```
JavaScript
let user2 = user1;
```

Like two people having the same house address.

```
Person A ⟶ House

Person B ⟶ House
```

If the house is painted blue:

```
Both see the blue house.
```

Because there is only one house.

Arrays Follow the Same Rule

```JavaScript
let arr1 = [1, 2, 3];

let arr2 = arr1;

arr2.push(4);

console.log(arr1);
```

Output:

```JavaScript
[1, 2, 3, 4]
```

Because both variables reference the same array.

Functions Are Also Stored in Heap

```JavaScript
function greet() {
    console.log("Hello");
}
```

Memory:

```
Stack                  Heap

greet ──────────────► Function Object
```

Functions are special objects in JavaScript.

Important Interview Question

Are Primitive Data Types Stored in Stack?

At a beginner level:

Yes.

Primitive values are stored directly in Stack memory.

Are Objects Stored in Heap?

Yes.

Objects, arrays, and functions are stored in Heap memory, while variables hold references.

Why Did We Learn This?

Because it explains:

Why this works:

```JavaScript
let a = 10;
let b = a;

b = 20;
```

```
a = 10
b = 20
```

But this behaves differently:

```JavaScript
let obj1 = { name: "Krishna" };
let obj2 = obj1;

obj2.name = "Rahul";
```

```
obj1.name = "Rahul"
obj2.name = "Rahul"
```

Exam-Oriented Definition

Stack Memory is used to store primitive data types and function call information. It stores values directly and provides fast access.

Heap Memory is used to store objects, arrays, and functions. Variables store references to data located in the heap.

Quick Revision

```
Primitive Types
↓
Stored directly
↓
Stack Memory
↓
Copy by Value
```

```
Objects, Arrays, Functions
↓
Stored in Heap
↓
Reference stored in Stack
↓
Copy by Reference
```

Practice Questions

Predict the Output

```JavaScript
```

```
let a = 5;
let b = a;

b = 10;

console.log(a);
```

Answer:

```
5
```

Predict the Output

```
JavaScript
let obj1 = { x: 10 };
let obj2 = obj1;

obj2.x = 20;

console.log(obj1.x);
```

Answer:

```
20
```

Key Takeaway

✅ Primitive values → stored directly, copied by value.
✅ Objects, Arrays, Functions → stored in heap, copied by reference.
✅ Stack memory is fast and stores primitive values.
✅ Heap memory stores complex data structures.
✅ Understanding Stack vs Heap helps explain many JavaScript behaviors.

Next Topic:

Strings in JavaScript (Methods, Properties, Template Literals, and String Manipulation).